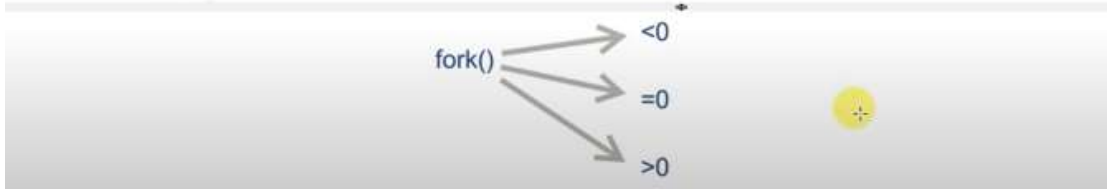
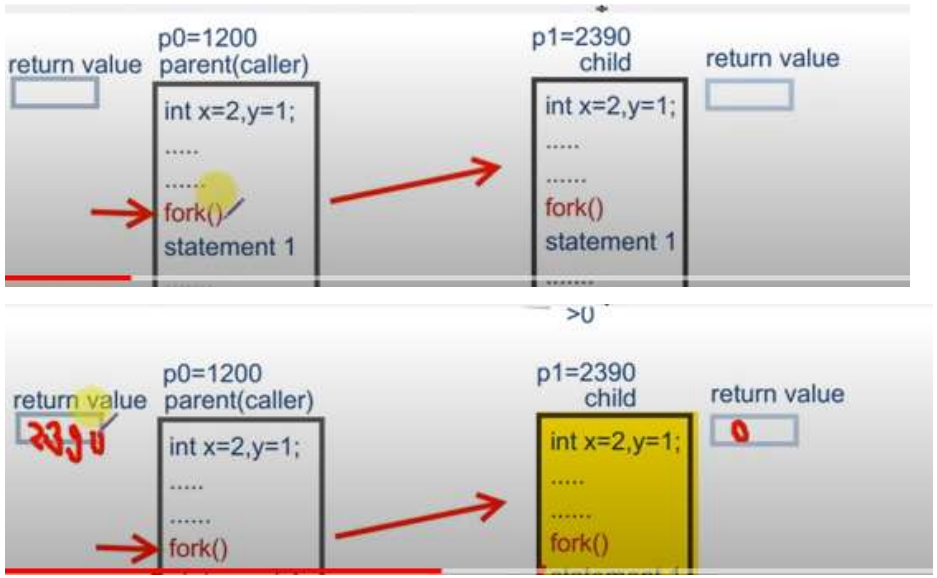
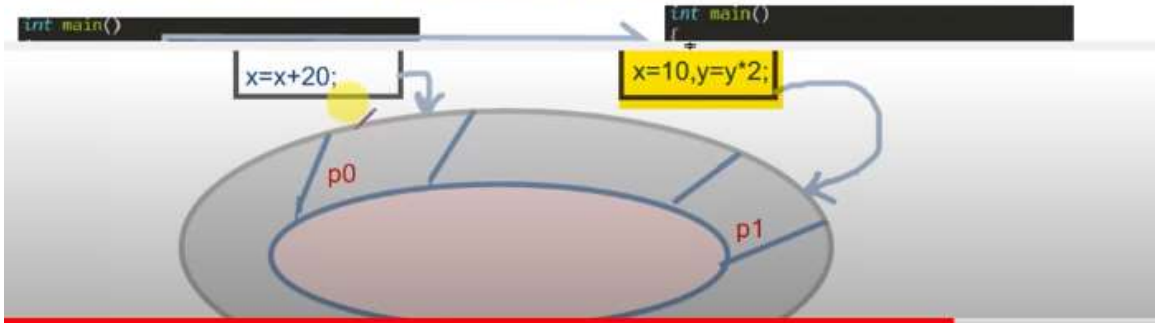
System call fork() is used to create processes. It takes no arguments and returns a process ID. The purpose of fork() is to create a new process, which becomes the child process of the caller. After a new child process is created, both processes will execute the next instruction following the fork() system call.

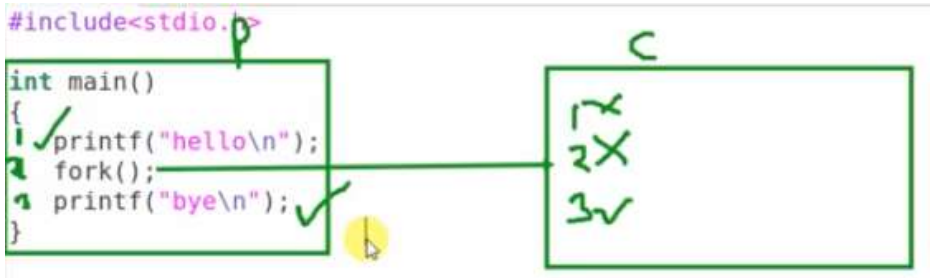
- To create child process we use fork(). fork() returns :
  - <0 fail to create child (new) process
  - =0 for child process
  - >0 i.e process ID of the child process to the parent process. When >0 parent

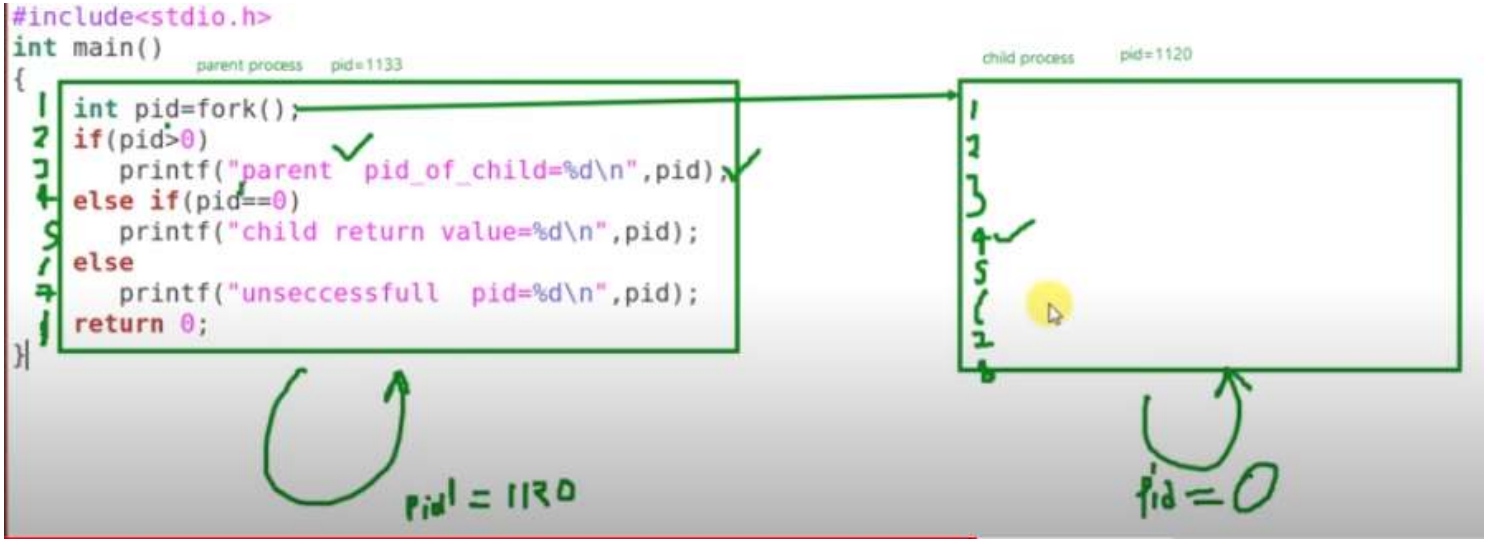



return value
p0=1200
parent(caller)
int x=2,y=1;
.....
.....
fork()
statement 1
p1=2390
child
int x=2,y=1;
.....
.....
fork()
statement 1
return value
>0
return value
p0=1200
parent(caller)
int x=2,y=1;
.....
.....
fork()
p1=2390
child
int x=2,y=1;
.....
.....
fork()
return value
0

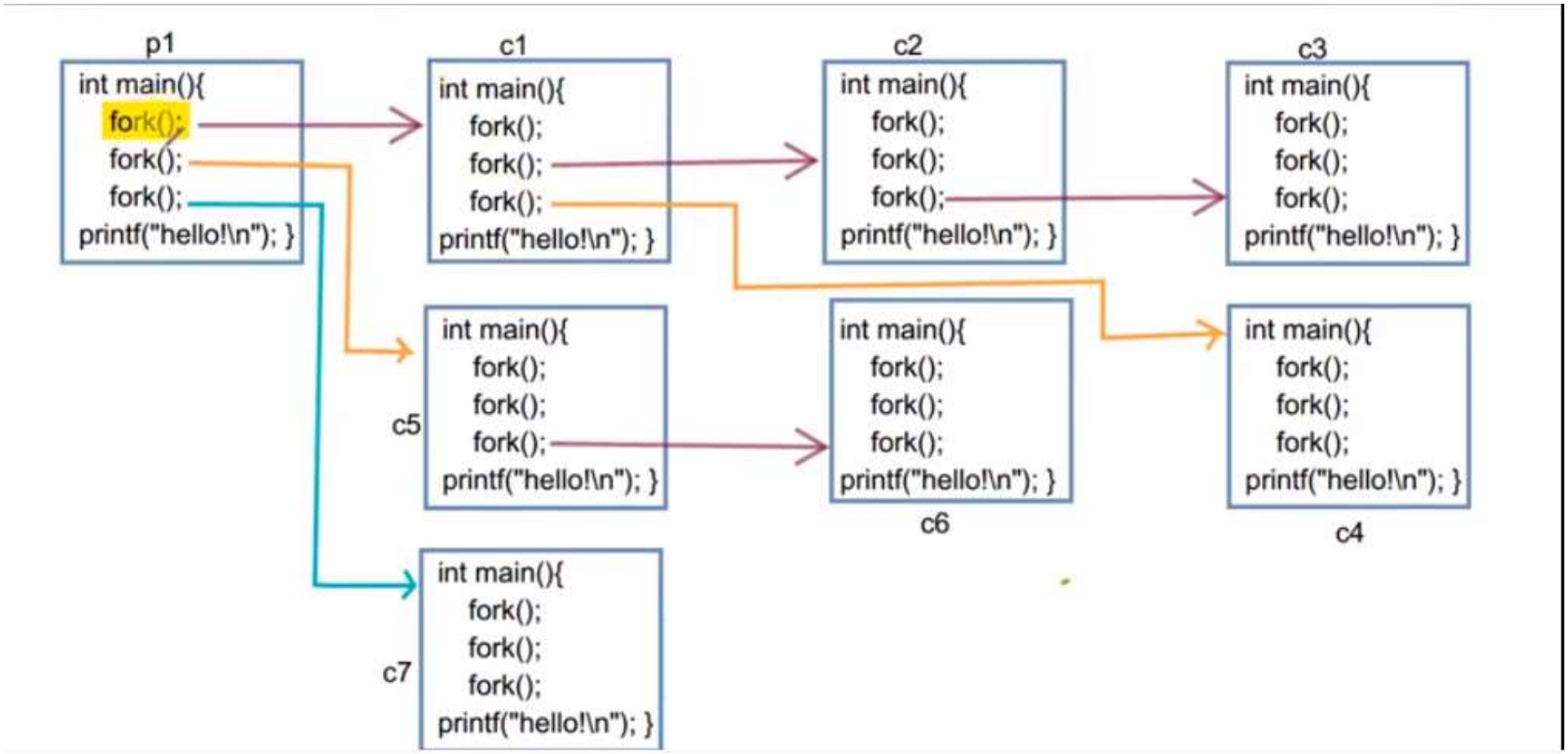
Since both processes have identical but separate address spaces, those variables initialized before the fork() call have the same values in both address spaces. Since every process has its own address space, any modifications will be independent of the others. In other words, if the parent changes the value of its variable, the modification will only affect the variable in the parent process's address space. Other address spaces created by fork() calls will not be affected even though they have identical variable names

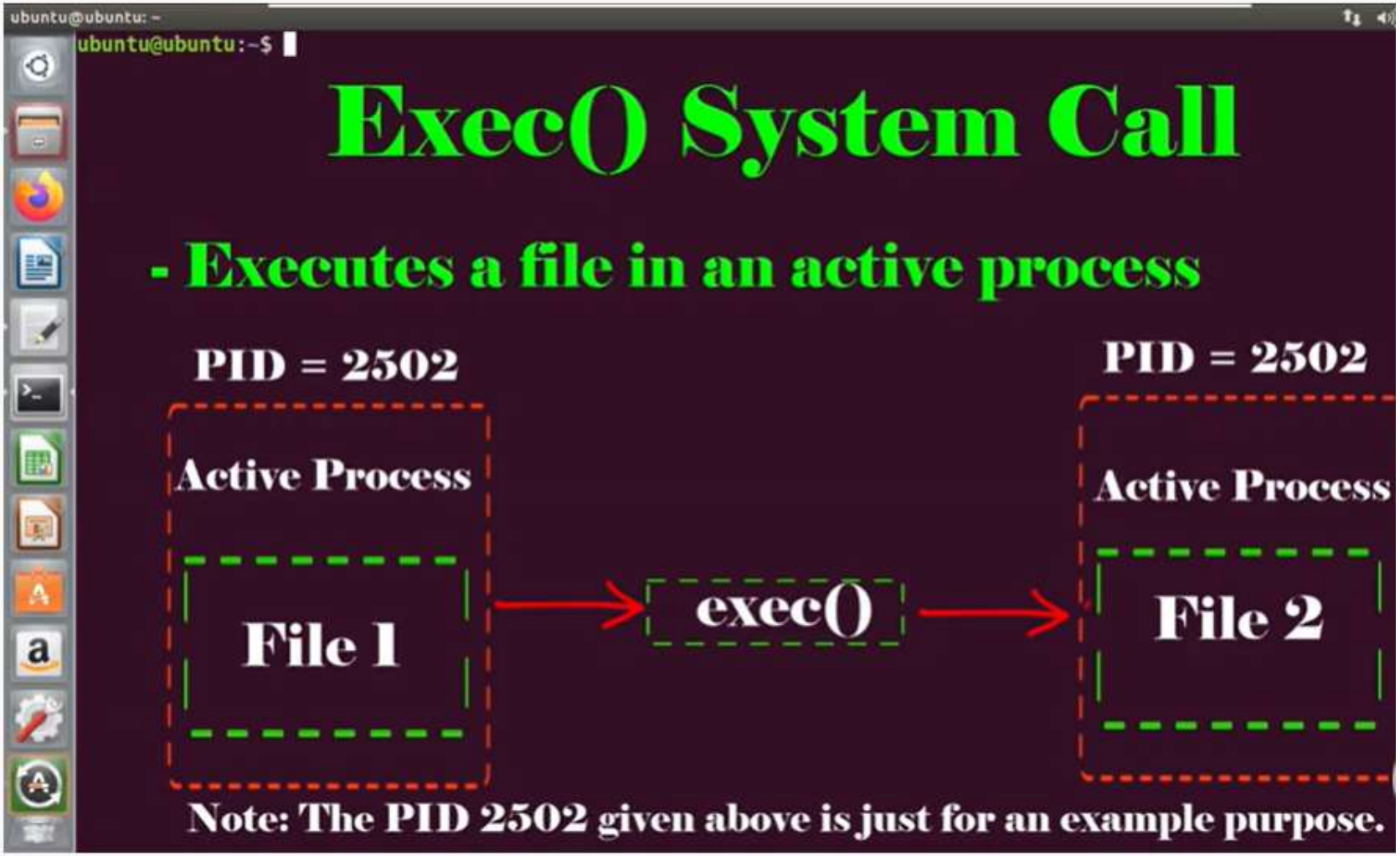


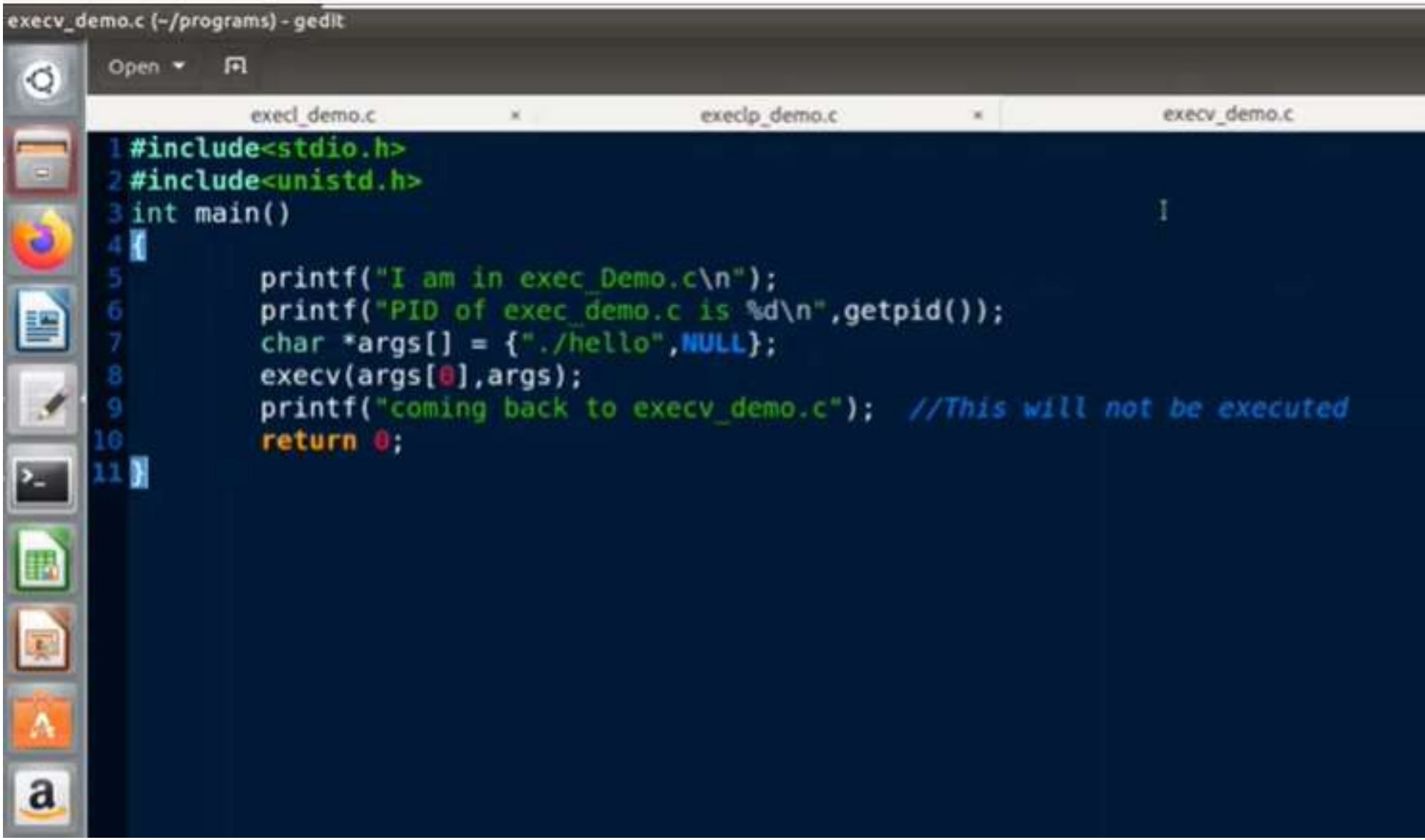

#include<stdio.h>
int main()
{
printf("hello\n");
fork();
printf("bye\n");
}
P
C
1
2
3

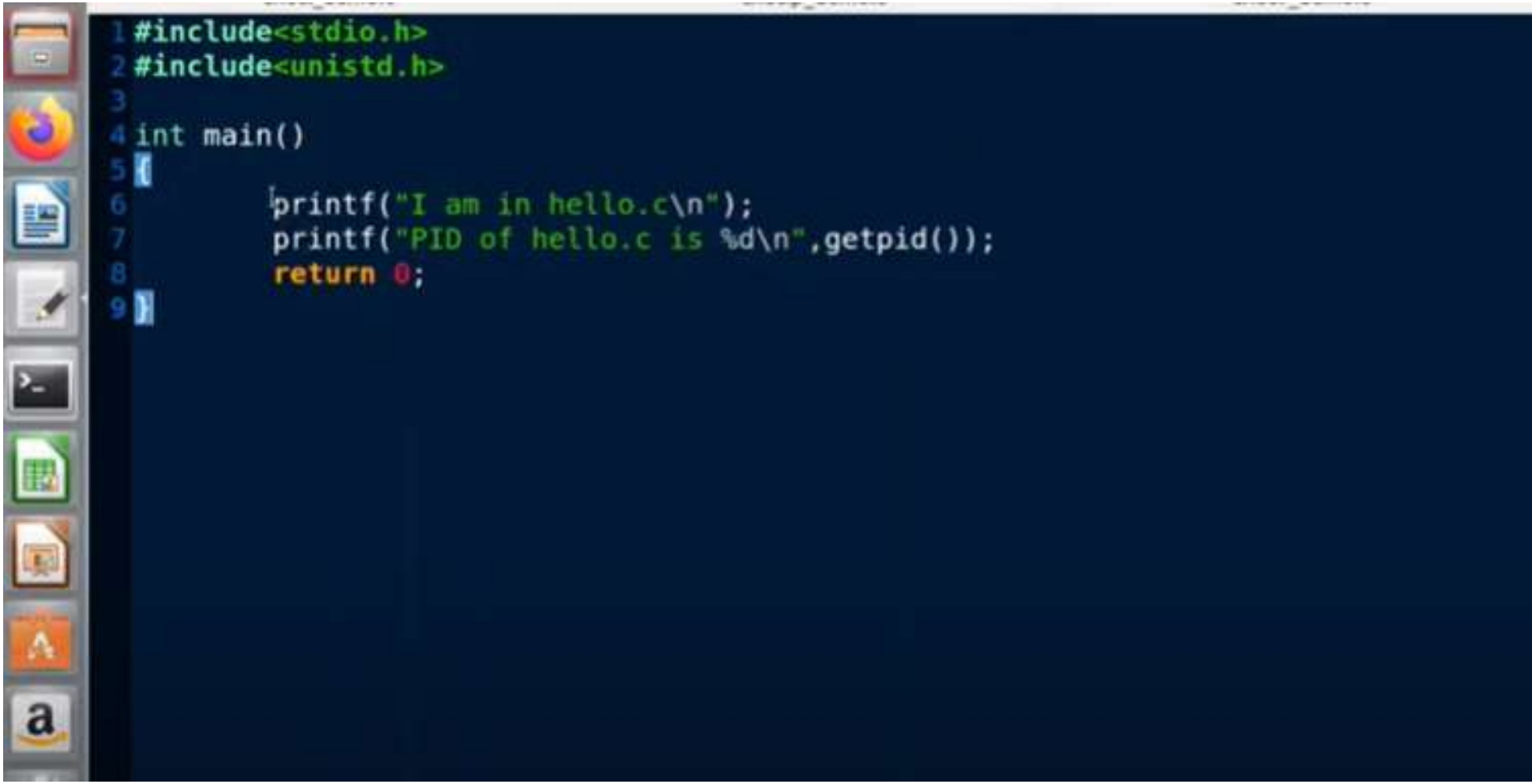

#include<stdio.h>
int main()
{
int pid=fork();
if(pid>0)
printf("parent pid_of_child=%d\n",pid);
else if(pid==0)
printf("child return value=%d\n",pid);
else
printf("unseccessfull pid=%d\n",pid);
return 0;
}
parent process pid=1133
child process pid=1120
pid = 1120
pid = 0

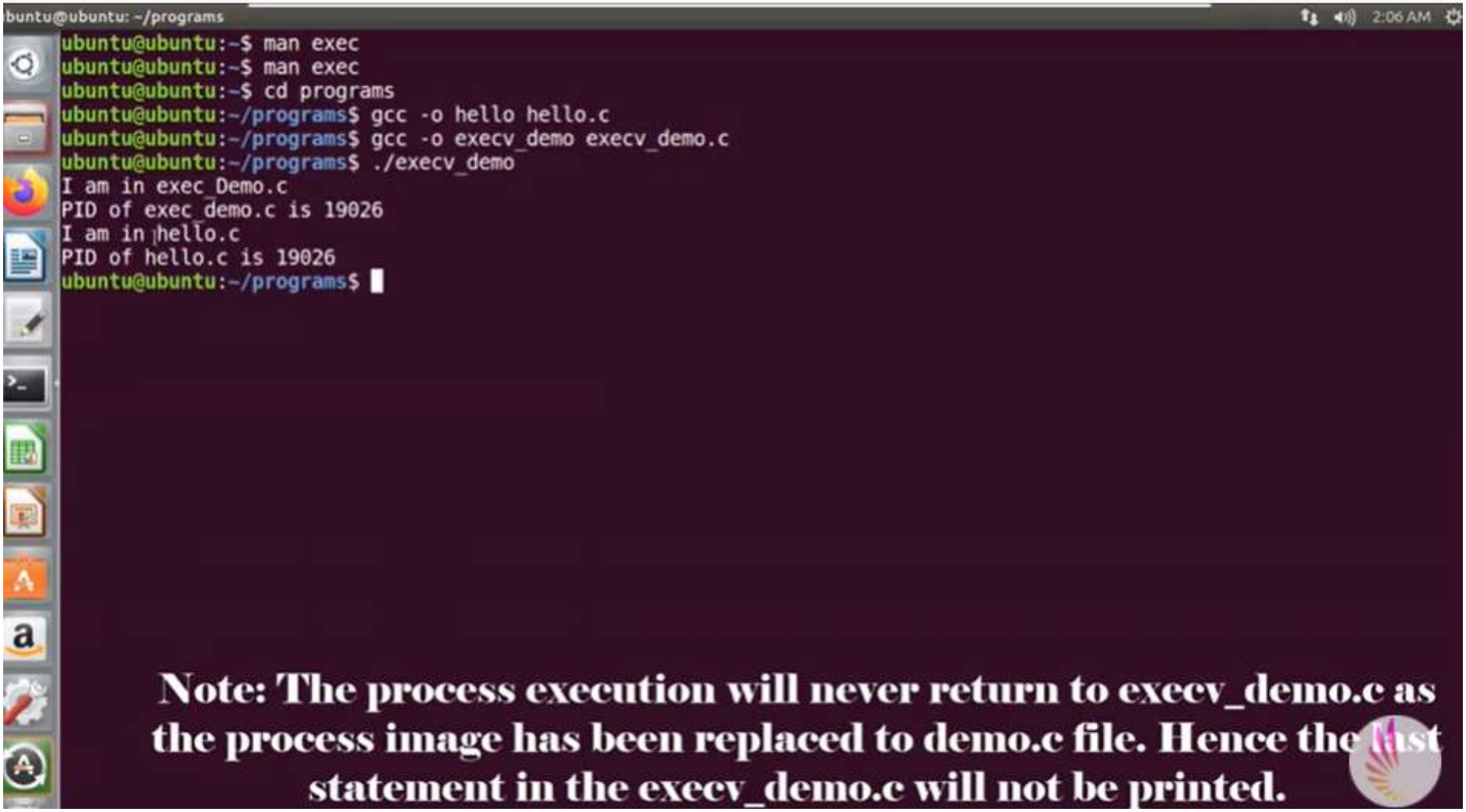
```c
#include<stdio.h>
int main()
{
    int pid=fork();
    if(pid>0)
       printf("parent  pid_of_child=%d\n",pid);
    else if(pid==0)
{
       printf("child return value=%d\n",pid);
       printf("child actual process id=%d",getpid());
}
    else
       printf("unseccessfull  pid=%d\n",pid);
    return 0;
}
```


p1
int main(){
fork();
fork();
fork();
printf("hello!\n"); }
c1
int main(){
fork();
fork();
fork();
printf("hello!\n"); }
c2
int main(){
fork();
fork();
fork();
printf("hello!\n"); }
c3
int main(){
fork();
fork();
fork();
printf("hello!\n"); }
c5
int main(){
fork();
fork();
fork();
printf("hello!\n"); }
c6
int main(){
fork();
fork();
fork();
printf("hello!\n"); }
c4
int main(){
fork();
fork();
fork();
printf("hello!\n"); }
c7


ubuntu@ubuntu: ~
ubuntu@ubuntu:~$
Exec() System Call
- Executes a file in an active process
PID = 2502
Active Process
File 1
exec()
PID = 2502
Active Process
File 2
Note: The PID 2502 given above is just for an example purpose.


execv_demo.c (~/programs) - gedit
Open
execl_demo.c
execlp_demo.c
execv_demo.c
#include<stdio.h>
#include<unistd.h>
int main()
{
printf("I am in exec_Demo.c\n");
printf("PID of exec_demo.c is %d\n",getpid());
char *args[] = {"./hello",NULL};
execv(args[0],args);
printf("coming back to execv_demo.c"); //This will not be executed
return 0;
}


#include<stdio.h>
#include<unistd.h>
int main()
{
printf("I am in hello.c\n");
printf("PID of hello.c is %d\n",getpid());
return 0;
}


buntu@ubuntu: ~/programs
2:06 AM
ubuntu@ubuntu:~$ man exec
ubuntu@ubuntu:~$ man exec
ubuntu@ubuntu:~$ cd programs
ubuntu@ubuntu:~/programs$ gcc -o hello hello.c
ubuntu@ubuntu:~/programs$ gcc -o execv_demo execv_demo.c
ubuntu@ubuntu:~/programs$ ./execv_demo
I am in exec_Demo.c
PID of exec_demo.c is 19026
I am in hello.c
PID of hello.c is 19026
ubuntu@ubuntu:~/programs$
Note: The process execution will never return to execv_demo.c as the process image has been replaced to demo.c file. Hence the last statement in the execv_demo.c will not be printed.